

ECE-GY 6143: Introduction to Machine Learning

Final Exam Solutions, Fall 2020

Prof. Sundeep Rangan

- This exam is done at home.
- You may use the Internet, your computer, class notes, labs, etc.
- You may *not* speak to others for help.
- Scan your solutions, convert to PDF and submit on Gradescope.
- Leave time for the submission.

Best of luck!

1. *SVM*. An SVM classifier is trained on 4×4 images, $\mathbf{X}_i$ with binary labels $y_i = \pm 1$. Three of the images in the training data set are:

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

with labels $y_1 = 1, y_2 = 1, y_3 = -1$. You are given a test image,

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

- (a) The training and test images, $\mathbf{X}_i$ and $\mathbf{X}$, are converted to 16-dimensional vectors $\mathbf{x}_i$ and $\mathbf{x}$. Find the squared distances

$$d_i = \|\mathbf{x} - \mathbf{x}_i\|^2, \quad i = 1, 2, 3.$$

- (b) Suppose a kernel is

$$K(\mathbf{x}, \mathbf{x}_i) = \max\{0, 1 - \|\mathbf{x}_i - \mathbf{x}\|^2 \gamma\}.$$

What value of $\gamma > 0$ guarantees that $K(\mathbf{x}, \mathbf{x}_i) > 0$ for $y_i = 1$ and $K(\mathbf{x}, \mathbf{x}_i) = 0$ for $y_i = -1$ for $i = 1, 2, 3$.

- (c) Complete the following python function,

```
def predict(Xtr,ytr,X,...):
    """
    Parameters
    Xtr: Training data, shape (ntr,nrow,ncol)
    ytr: Training labels, shape (ntr,)
    X: Test data, shape (n,nrow,ncol)
    ... Add other parameters as needed

    Returns:
    yhat: Predicted labels, shape (n,)
    """
    ...
    return yhat
```

to perform the SVM classification,

$$z = \sum_{i=1}^n \alpha_i y_i K(x, x_i) + b, \quad \hat{y} = \begin{cases} 1, & \text{if } z > 0 \\ -1, & \text{otherwise.} \end{cases}$$

For full credit, use python broadcasting.

Solution:

(14 points)

- (a) (5 points) First, we compute the differences of the images. For example, the difference for the first image is

$$\mathbf{X} - \mathbf{X}_1 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -0.5 & 0.5 & 0 & 0 \\ -1 & 1 & 0 & 0 \end{bmatrix}.$$

Therefore, the squared distance is,

$$d_1 = \|\mathbf{x} - \mathbf{x}_1\|^2 = 2(0.5)^2 + 2(1)^2 = 2.5,$$

since there are two pixels where the squared differences are $(0.5)^2$ and two with $(1)^2$. Similarly, we obtain,

$$d_2 = 2(0.5)^2 + 4(1)^2 = 4.5$$

$$d_3 = 2(0.5)^2 + 7(1)^2 = 7.5.$$

- (b) (4 points) You can see that $K(\mathbf{x}, \mathbf{x}_i) = 0$ when

$$1 - \gamma\|\mathbf{x} - \mathbf{x}_i\|^2 \geq 0 \Leftrightarrow 1 - \gamma d_i \geq 0 \Leftrightarrow \gamma \leq \frac{1}{d_i}.$$

We want $K(\mathbf{x}, \mathbf{x}_i) > 0$ for $i = 1, 2$, and therefore,

$$\gamma < \frac{1}{d_1} = \frac{1}{2.5} = 0.4$$

$$\gamma < \frac{1}{d_2} = \frac{1}{4.5} = 0.22.$$

Also, we want $K(\mathbf{x}, \mathbf{x}_i) > 0$ for $i = 3$, and therefore,

$$\gamma \geq \frac{1}{d_3} = \frac{1}{7.5} = 0.13.$$

Hence, $\gamma \in [0.13, 0.22)$. If you provide only a single γ in this range, you will get 2 points.

- (c) (5 points) One solution is

```
def predict(Xtr, ytr, alpha, b, X):

    # Compute distances D[i,j,k1,k2] = X[i,k1,k2] - Xtr[j,k1,k2]
    D = X[:,None,:,:] - Xtr[None,:,:,:]

    # Square the differences
```

```

Dsq = np.sum(D**2, axis=(2,3) )

# Apply the kernel
K = np.maximum(0, 1-Dsq*gamma)

# Compute the decision function
z = K.dot(alpha*ytr)
yhat = (z > 0).astype(int)

return yhat

```

You will lose 2 points for not using broadcasting.

2. *Neural Networks.* Consider a neural network, with $N_i = 2$ input units, $\mathbf{x} = (x_1, x_2)$, $N_h = 3$ hidden units with ReLU activations and one output unit for regression,

$$z_j^H = \sum_{k=1}^{N_i} W_{jk}^H x_k + b_j^H, \quad u_j^H = \max\{0, z_j^H\}, \quad j = 1, \dots, N_h$$

$$\hat{y} = \sum_{k=1}^{N_h} W_k^O u_k^H + b^O,$$

- (a) Suppose that

$$\mathbf{W}^H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}, \quad \mathbf{b}^H = \begin{bmatrix} -0.5 \\ -1 \\ 3 \end{bmatrix}$$

Write equations for z_j^H in terms of x for $j = 1, 2, 3$.

- (b) Draw the region of inputs (x_1, x_2) where $u_j^H > 0$ for all j .
- (c) You are given data $\mathbf{x}, \mathbf{y}$ as well as weights and biases $\mathbf{w}_h, \mathbf{b}_h$ for the hidden layer. Write a few lines of python code to fit $\mathbf{w}_o, \mathbf{b}_o$ for the output layer by minimizing the MSE. You may assume you have a function

```
beta = lstsq(A,b) # Solves the least squares problem A.dot(beta) = b
```

Solution:

(14 points)

- (a) (3 points) We have

$$\mathbf{z}^H = \mathbf{W}^H \mathbf{x} + \mathbf{b}^H = \begin{bmatrix} x_1 - 0.5 \\ x_2 - 1 \\ -x_1 - x_2 + 3 \end{bmatrix}.$$

- (b) (5 points) Since $u_j^H = \max\{0, z_j^H\}$, we have $u_j^H > 0$ if and only if $z_j^H > 0$. Using the expressions for z_j^H in part (a),

$$u_1^H > 0 \iff x_1 > 0.5$$

$$u_2^H > 0 \iff x_2 > 1$$

$$u_3^H > 0 \iff x_1 + x_2 < 3.$$

The boundaries of the three regions are shown in Fig. 1. Note that for the last constraint, the intercepts are at $x_1 = 3$ and $x_2 = 3$.

- (c) (6 points) One solution is:

```
# Compute the hidden outputs
Zhid = X.dot(WH.T) + bh[None,:]
Uhid = np.maximum(0, Zhid)
```

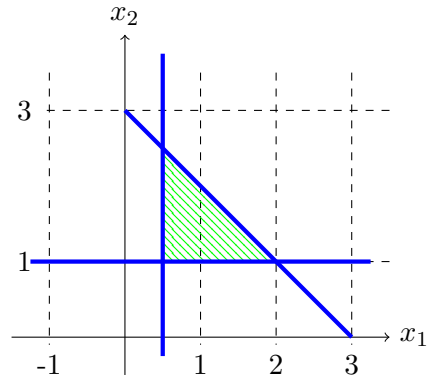


Figure 1: Blue lines: Boundaries of the three hidden units. Green region: The region where all the hidden units $u_j^H > 0$.

```
# Create the data matrix
# A = [Uhid 1]
n,p = Uhid.shape
A = np.hstack((Uhid, np.ones(n)))

# Solve the LS problem:  A.dot(beta) = y
beta = lstsq(A,y)

# Extract the output vector and bias
Wo = beta[:p]
bo = beta[p]
```

3. *Backpropagation.* Consider the model with an M -dimensional input $\mathbf{x} = (x_1, \dots, x_M)$ and scalar output $\hat{y}$ given by

$$z_k = \sum_{j=1}^M A_{kj}x_j + b_k, \quad u_k = \frac{e^{z_k}}{\sum_{\ell=1}^K e^{z_\ell}}, \quad k = 1, \dots, K$$

$$\hat{y} = \sum_{k=1}^K \sum_{j=1}^M C_{kj}x_j u_k + \sum_{k=1}^K d_k u_k$$

with parameters A_{kj} , b_k , C_{kj} and d_k .

- (a) You are given training samples $\mathbf{x}_i = (x_{i1}, \dots, x_{iM})$ and y_i . Write the equations relating the inputs x_{ij} to the output $\hat{y}_i$.
- (b) Draw the computation graph between the data, parameters and loss function using the loss function,

$$J = \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

- (c) Show how to compute the components of $\partial J / \partial z$ from the components of $\partial J / \partial u$.

Solution:

(14 points)

- (a) (4 points) We add the index i as follows:

$$z_{ik} = \sum_{j=1}^M A_{kj}x_{ij} + b_k, \quad u_{ik} = \frac{e^{z_{ik}}}{\sum_{\ell=1}^K e^{z_{i\ell}}}, \quad k = 1, \dots, K$$

$$\hat{y}_i = \sum_{k=1}^K \sum_{j=1}^M C_{kj}x_{ij}u_{ik} + \sum_{k=1}^K d_k u_{ik}.$$

- (b) (5 points) See Fig. 2.

- (c) (5 points) We have

$$\frac{\partial u_{im}}{\partial z_{ik}} = \frac{1}{(\sum_{\ell} e^{z_{i\ell}})^2} \left[e^{z_{im}} \delta_{km} \sum_{\ell} e^{z_{i\ell}} - e^{z_{ik}} e^{z_{im}} \right].$$

By chain rule,

$$\frac{\partial J}{\partial z_{ik}} = \sum_{m=1}^K \frac{\partial J}{\partial u_{im}} \frac{\partial u_{im}}{\partial z_{ik}}.$$

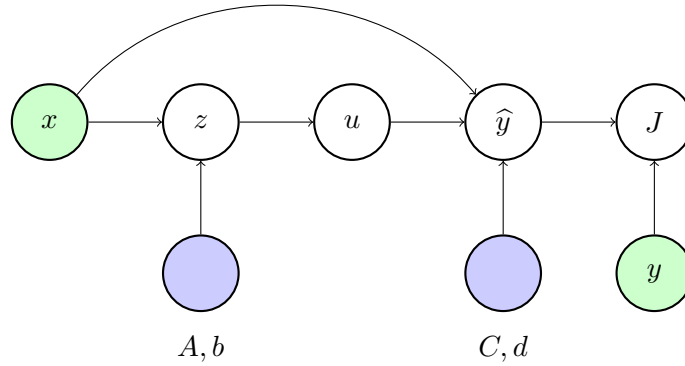


Figure 2: Computation graph for Problem 3. Parameters are shown in light blue and data in light green.

4. *CNN dimensions.* A neural network for image processing has the following input and first two layers:
- **Input:** 128×256 RGB image
 - **Conv1:** 2D convolution, $K = 9 \times 9$ filters, $N_o = 64$ output channels, same size output, ReLU activation
 - **MaxPool1:** 2D max pooling, $K = 4 \times 4$ pool size, horizontal and vertical stride $s = 2$.
- (a) What are the dimensions of the input and the outputs for the first two layers for a mini-batch of 32 images.
- (b) How many parameters are there in the **Conv1** and **MaxPool1** layers?
- (c) If the value of only the red channel on a single pixel of an input sample is changed, what is the maximum number of output values that could be changed in the outputs of the **Conv1** and **MaxPool1** layers?

Solution:

(14 points)

(a) (4 points) The dimensions would be:

- **Input:** $(32, 128, 256, 3)$
- **Conv1:** Since it has same size and 64 output channels, its size will be $(32, 128, 256, 64)$.
- **MaxPool1:** The stride of 2 will reduce the horizontal and vertical by 2 resulting in an output size $(32, 64, 128, 64)$.

(b) (4 points) The **Conv1** layer has $(9)(9)(3)(64)$ weights and 64 biases. The **MaxPool1** has no parameters.

(c) (6 points) If the value of one input channel in one pixel is changed:

- **Conv1:** Every pixel will affect a 9×9 region in all output channels. Note that even a single input channels can affect all output channels, so the total effect can

be up to a change of $(9)(9)(64)$ values in the output of `Conv1`.

- `MaxPool1`: The pooling can be thought of as a filtering by a 4×4 kernel, followed by a down sampling by 2×2 . The filtering will increase the 9×9 region to 13×13 . The down sampling by 2×2 will reduce this to 7×7 . So, the change will affect $(7)(7)(64)$ values in the output of `MaxPool1`.

5. *Transfer learning.* You want to build a classifier that detects bicycles in images. In each sample, the input to the classifier is a set of 5 photos of some scene taken at different angles. Each photo is an RGB image of size 128×128 . The output of the classifier is a single binary value, 0 or 1, for each sample, indicating if there was a bicycle in *any* of the five photos in the sample.

- (a) You want to write the classifier that takes a set of samples `x` and return a set of predictions `yhat`. What should the shape of `x` and `yhat` be?
- (b) You are given an excellent pre-trained image classifier that works on a single 128×128 RGB image. The pre-trained classifier was trained on 1000 image classes and can be accessed by:

```
Z = pretrained.predict(X)  # Outputs the logits for the 1000 classes
```

Suppose that in the pre-trained classifier, bicycles were one of the image classes, say class 50. Write a possible implementation of a classifier using the pretrained model:

```
def classify(X, pretrained):
    ...
    return yhat
```

There is no single correct answer. Do something reasonable with an explanation.

- (c) Now suppose that bicycles were *not* one of the image classes that the pretrained classifier uses. So, you decide to transfer learn using the outputs of one of the layers near the end of the pre-trained model using simple logistic regression on those outputs. Assume you have the following functions:

```
# Outputs the 2000 hidden units at some layer near the
# end of the pre-trained model
Z = pretrain_base.predict(X)

reg = LogisticRegression() # Builds a logistic regression object
reg.fit(X, y)              # Fits the logistic model from X to y
yhat = reg.predict(X)      # Predicts binary outputs
```

Write functions to fit and predict the pre-trained model:

```
def fit(Xtr, ytr, ...)
    ...
    return ...

def predict(X, ...)
    ...
    return yhat
```

The functions should take whatever inputs and outputs are necessary.

Solution:

(15 points)

- (a) (3 points) The shape of the input x should be $(n, 128, 128, 5)$ and the shape of the output should be $(n,)$.
- (b) (5 points) One possible solution is:

```
def classify(X, pretrained):
    # Get dimensions
    nimage = X.shape[-1]

    # Compute the maximum logit for the bicycle class
    # across the images
    for i in range(nimage):
        # Run image i through the pretrained model
        Z = pretrained.predict(X[:, :, :, i])

        # Get the logit for the bicycle class
        z0 = Z[:, 50]

        # Take the maximum
        if i == 0:
            zmax = z0
        else:
            zmax = np.maximum(zmax, z0)

    # Make the decision on the maximum logit
    yhat = (zmax > 0).astype(int)

    return yhat
```

If you make minor errors like forgetting the conversion `astype(int)` or use `np.max` instead of `np.maximum`, you will not be penalized marks.

- (c) (7 points) For the fitting we stack the 2000 outputs from all 5 images creating a 10000-dimensional feature vector. Then we train on the 10000 feature vector

```
def fit(Xtr, ytr, pretrained_base):
    # Get dimensions
    nimage = Xtr.shape[-1]

    # Get the hidden outputs from each image
    for i in range(nimage):
        # Run image i through the pretrained model
        Z = pretrained_base.predict(Xtr[:, :, :, i])
```

```

# Stack the ouptuts
if i == 0:
    Zstack = Z
else:
    Zstack = np.hstack((Zstack,Z))

# Fit the logistic model
reg = LogisticRegression()
reg.fit(Zstack, ytr)

# Return the model
return reg

```

For the prediction function, we create the stack values as before and then run the stacked values through the logistic model prediction:

```

def predict(X,reg,pretrained_base):
    # Get dimensions
    nimage = X.shape[-1]

    # Get the hidden outputs from each image
    for i in range(nimage):
        # Run image i through the pretrained model
        Z = pretrained_base.predict(X[:, :, :, i])

        # Stack the ouptuts
        if i == 0:
            Zstack = Z
        else:
            Zstack = np.hstack((Zstack,Z))

    # Predict using the logistic model
    yhat = reg.predict(Zstack)
    return yhat

```

6. *PCA*. Let $\mathbf{X}$ is a data matrix of $n = 100$ samples, each of dimension $p = 200$ with one sample per row. Let $\tilde{\mathbf{X}}$ be the matrix with the mean μ removed from the each row. Suppose it has a truncated SVD of the form,

$$\frac{1}{\sqrt{n}}\tilde{\mathbf{X}} \approx \mathbf{U}\text{diag}(\mathbf{s})\mathbf{V}^T,$$

where

$$\mathbf{U} = \begin{bmatrix} 0.5 & 0.3 & 0.1 \\ 0.6 & 0.4 & 0.2 \\ \vdots & \vdots & \vdots \end{bmatrix}, \quad \mathbf{s} = [10, 8, 2],$$

The columns of $\mathbf{U}$ and $\mathbf{V}$ are orthonormal. Only the first two rows of $\mathbf{U}$ are shown. Let $\mathbf{v}_j$, $j = 1, \dots, 3$ be the three columns of $\mathbf{V}$.

- What are the dimensions of $\mathbf{U}$ and $\mathbf{V}$?
- Suppose $\mathbf{x}_1$ is the first row of $\mathbf{X}$. Write $\mathbf{x}_1$ as a linear combination of the mean μ and the three PCs $\mathbf{v}_j$, $j = 1, 2, 3$.
- Suppose that $\hat{\mathbf{x}}_1$ is the approximation of $\mathbf{x}_1$ constructed using only μ , and the first PC $\mathbf{v}_1$. What is $\|\hat{\mathbf{x}}_1 - \mathbf{x}_1\|^2$?

Solution:

(14 points)

(a) (4 points) $\mathbf{U}$ has shape 100×3 and $\mathbf{V}$ has shape 200×3 .

(b) (5 points) The coefficients along the PCs are:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V} = \sqrt{n}\mathbf{U}\text{diag}(\mathbf{s})\mathbf{V}^T\mathbf{V} = \sqrt{n}\mathbf{U}\text{diag}(\mathbf{s}).$$

The first row of the coefficient matrix will be,

$$\begin{aligned} \mathbf{z}_1 &= \sqrt{n}\mathbf{u}_1\text{diag}(\mathbf{s}) \\ &= \sqrt{100}[0.5, 0.3, 0.1]\text{diag}([10, 8, 2]) = [50, 24, 2]. \end{aligned}$$

Hence,

$$\mathbf{x}_1 = \mu + 50\mathbf{v}_1 + 24\mathbf{v}_2 + 2\mathbf{v}_3.$$

(c) (5 points) The approximation with one PC is

$$\hat{\mathbf{x}}_1 = \mu + 50\mathbf{v}_1.$$

Hence, the error is,

$$\mathbf{x}_1 - \hat{\mathbf{x}}_1 = 24\mathbf{v}_2 + 2\mathbf{v}_3.$$

Since the PCs are orthonormal, the squared error will be

$$\|\mathbf{x}_1 - \hat{\mathbf{x}}_1\|^2 = (24)^2 + 2^2.$$

7. *K-means*. You are given training data $(\mathbf{x}_i, y_i)$ with continuous scalar-valued outputs y_i . To train the model, you decide to:

- Perform *K-means* on the training data $\mathbf{x}_i$ resulting in cluster centers $\boldsymbol{\mu}_k$, $k = 1, \dots, K$.
- Compute $\bar{y}_k$, the mean of the y_i 's in each cluster k .

Then, given a new input $\mathbf{x}$ you make a prediction:

$$\hat{y} = \frac{\sum_{k=1}^K \bar{y}_k e^{-\gamma \|\mathbf{x} - \boldsymbol{\mu}_k\|^2}}{\sum_{k=1}^K e^{-\gamma \|\mathbf{x} - \boldsymbol{\mu}_k\|^2}}$$

The number of clusters K and the scaling value $\gamma > 0$ are fixed (they are not trained). In the questions below, you can assume you have the following functions:

```
km = KMeans(n_cluster=nc)    # Creates a K-Means object
km.fit(X)                    # Fits the k-means clusters
km.predict(X)                # Returns the index of the closest cluster for each row in X
km.cluster_centers_          # Returns the cluster centers (one cluster per row)
```

- (a) Suppose a test sample $\mathbf{x}$ is close to samples in a cluster of four points with outputs $y_i = 4, 4, 5, 6$. The sample $\mathbf{x}$ is far from the other cluster centers. Approximately, what is $\hat{y}$? For full marks, provide an explanation.
- (b) Write a function to perform the training:

```
def fit(Xtr, ytr, ...)
    ...
    return ...
```

The function should take whatever arguments are necessary and return the relevant parameters in the model.

- (c) Write a function to compute a vector of values `yhat` on new data `X`.

```
def predict(X, ...)
    ...
    return yhat
```

Add whatever input arguments are needed.

Solution:

(15 points)

- (a) (4 points) If the sample $\mathbf{x}$ is close to some cluster $\boldsymbol{\mu}_k$, then, for all $j \neq k$,

$$\|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \ll \|\mathbf{x} - \boldsymbol{\mu}_j\|^2 \Rightarrow e^{-\|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \gamma} \gg e^{-\|\mathbf{x} - \boldsymbol{\mu}_j\|^2 \gamma}.$$

Hence, the predicted value at $\mathbf{x}$ will be:

$$\begin{aligned}\hat{y} &= \frac{\sum_{k=1}^K e^{-\gamma\|\mathbf{x}-\boldsymbol{\mu}_k\|^2} \bar{y}_k}{\sum_{k=1}^K e^{-\gamma\|\mathbf{x}-\boldsymbol{\mu}_k\|^2}} \\ &\approx \frac{e^{-\gamma\|\mathbf{x}-\boldsymbol{\mu}_k\|^2} \bar{y}_k}{e^{-\gamma\|\mathbf{x}-\boldsymbol{\mu}_k\|^2}} = \bar{y}_k.\end{aligned}$$

So it is close to the average value of the labels in the cluster. In this case,

$$\hat{y} \approx \bar{y}_k = \frac{1}{4}[4 + 4 + 5 + 6] = 4.75.$$

You will receive only 2 points if you provide this answer without an explanation.

(b) (6 points) One solution is as follows:

```
def fit(Xtr,ytr,nc):
    # Run k-means on the training data
    km = KMeans(n_cluster=nc)
    km.fit(Xtr)

    # Get cluster centers
    mu = km.cluster_centers_

    # Get the mean in each cluster
    ind = km.predict(Xtr)
    ymean = np.zeros(nc)
    for i in range(nc):
        I = np.where(ind == i)[0]
        ymean[i] = np.mean(ytr[I])

    return mu, ymean
```

(c) (5 points) One solution is as follows:

```
def predict(X,mu,ymean, gamma):

    # Find the distance to the cluster centers
    # D[i,j,:] = X[i,:] - mu[j,:]
    # Dsq[i,j] = sum(D[i,j,:]**2)
    D = X[:,None,:] - mu[None,:,:]
    Dsq = np.mean(D**2, axis=2)

    # Compute A[i,j] = exp(-gamma*Dsq[i,j])
    A = np.exp(-gamma*Dsq)
```

```
# Compute  $\hat{y}_i = \sum_j A[i,j] \bar{y}_j / \sum_j A[i,j]$ 
yhat = A.dot(ymean) / np.sum(A, axis=1)

return yhat
```